

A 70-Vertex Lower Bound for Cubic Bipartite Counterexamples to the Erdős–Gyárfás Conjecture

Draft manuscript – 17 August 2026

Abstract

The Erdős–Gyárfás conjecture asserts that every finite simple graph of minimum degree at least three contains a cycle whose length is a power of two. For cubic bipartite graphs, Tranquilli proved by certified exhaustive computation that every graph on at most 58 vertices contains a cycle of length 4, 8, or 16, giving a 60-vertex lower bound for a cubic bipartite counterexample.

We extend this finite frontier to 70 vertices: every simple cubic bipartite graph on at most 68 vertices contains a C_4 , C_8 , or C_{16} . The proof combines the theorem of O’Keefe and Wong that every cubic graph of girth 10 has at least 70 vertices with a triangle-rooted restricted-growth enumeration of linear symmetric v_3 -configurations on at most 34 points. We give a cap-independent coverage proof for the enumeration, prove the correctness of the C_8 and C_{16} pruning predicates, and exhaust both normalized triangle-rooted search trees at cap 34.

The two searches contain 225,745 states and examine 28,199,959 candidate extensions in total, with zero completions. The finite exhaustion is accompanied by static certificate streams. An independent verifier reconstructs the complete proposal schedule and checks explicit local witnesses for rejected branches. The cap-34 depth-first search was also rebuilt and rerun directly from a fresh checkout of the immutable upstream source, reproducing the exact counters and decision-transcript hashes.

1 Introduction

The Erdős–Gyárfás conjecture asks whether every finite simple graph of minimum degree at least three contains a cycle whose length is a power of two [1]. The conjecture remains open, including under substantial structural restrictions.

Here we consider the cubic bipartite case. Since every cycle in a bipartite graph has even length, a counterexample must in particular avoid C_4 , C_8 , and C_{16} . Tranquilli [3] recently established by certified exhaustive computation that every simple cubic bipartite graph on at most 58 vertices contains at least one of these three cycles. Equivalently, any cubic bipartite counterexample has at least 60 vertices.

The present work retains the same incidence-geometric viewpoint but pushes the triangle-rooted search to a point cap of 34. A C_4 -free cubic bipartite graph is the Levi graph of a linear symmetric v_3 -configuration, and a C_6 becomes a Berge triangle. The key structural observation is that below 70 vertices a relevant connected cubic bipartite graph must contain a C_6 : if it did not, its girth would be at least 10; the girth-10 case is excluded below 70 by O’Keefe and Wong [2], and girth at least 12 is excluded by the cubic Moore bound.

This reduces the problem to two normalized triangle-rooted search trees. Our main result is the following.

Theorem 1.1. *Every simple cubic bipartite graph G with*

$$|V(G)| \leq 68$$

contains a simple cycle of length 4, 8, or 16.

Corollary 1.2. *Any simple cubic bipartite counterexample to the Erdős–Gyárfás conjecture has at least 70 vertices.*

No claim is made in this paper about graphs on 70 or more vertices.

The proof is deliberately separated into three interfaces. First, a mathematical coverage theorem shows that every relevant target configuration occurs on at least one branch of the restricted-growth search. Second, the two cycle-pruning predicates are proved exact, so a target branch cannot be pruned falsely. Third, a static certificate independently accounts for the finite search tree: the verifier reconstructs the proposal schedule, checks local rejection witnesses, recursively follows accepted children, and reports zero completed configurations for both roots. This separation prevents a computational observation from silently substituting for a coverage or correctness argument.

2 Incidence configurations and Berge cycles

Let $G = (X, Y; E)$ be a connected cubic bipartite graph. Since

$$3|X| = |E| = 3|Y|,$$

we have $|X| = |Y| = v$. Regard the vertices of X as points. For each $y \in Y$, take its neighborhood $N(y) \subseteq X$ as a block. Every block contains three points and every point belongs to three blocks, so this gives a symmetric v_3 incidence structure

$$\mathcal{C} = (P, \mathcal{B}), \quad |P| = |\mathcal{B}| = v.$$

Its Levi graph is exactly G .

If G is C_4 -free, two distinct blocks cannot share two points. Thus the incidence structure is *linear*. In a linear incidence structure, a Berge cycle of length k consists of distinct points $p_0, \dots, p_{k-1}$ and distinct blocks $B_0, \dots, B_{k-1}$ such that

$$p_i \in B_{i-1} \cap B_i$$

for indices modulo k . It corresponds to the simple Levi cycle

$$p_0, B_0, p_1, B_1, \dots, p_{k-1}, B_{k-1}, p_0$$

of length $2k$. Hence

$$C_6 \longleftrightarrow \text{Berge triangle}, \quad C_8 \longleftrightarrow \text{Berge 4-cycle}, \quad C_{16} \longleftrightarrow \text{Berge 8-cycle}.$$

3 A Berge triangle is forced below 70 vertices

Lemma 3.1. *Let H be a connected cubic bipartite graph with $|V(H)| < 70$. If H contains no C_4 and no C_8 , then H contains a C_6 .*

Proof. Assume that H also contains no C_6 . Because H is bipartite, every cycle has even length, so the absence of C_4 , C_6 , and C_8 implies that the girth is at least 10.

If the girth is exactly 10, the theorem of O’Keefe and Wong [2] gives

$$|V(H)| \geq 70,$$

contrary to the hypothesis.

Suppose instead that the girth is at least 12. Choose an edge uv . Grow the two nonbacktracking cubic trees rooted at u and v , excluding the central edge. Up to depth 5 on each side, all encountered vertices must be distinct; a collision between vertices reached at distances a and b from the two ends of the central edge produces a cycle of length $a + b + 1 \leq 10$, contradicting the assumed girth. Therefore

$$|V(H)| \geq 2(1 + 2 + 4 + 8 + 16 + 32) = 126,$$

again impossible.

Thus H contains a C_6 . □

Remark 3.2. *The only input used from O’Keefe and Wong is the minimum-order statement $f(3, 10) = 70$. No classification of the cubic girth-10 graphs of order 70 is used.*

4 The two normalized triangle roots

Let $\mathcal{C}$ be a connected linear symmetric v_3 -configuration containing a Berge triangle. Relabel its triangle points as 0, 1, 2. After assigning new labels in first-occurrence order, the three triangle blocks can be written

$$\{0, 1, 3\}, \quad \{1, 2, 4\}, \quad \{0, 2, 5\}.$$

Consider the third block through point 0. Linearity prevents it from containing 1, 2, 3, or 5, since each of those points already occurs with 0 in a triangle block. Among the introduced labels, only 4 can therefore be reused. The third block through 0 is consequently either

$$\{0, 4, 6\}$$

or

$$\{0, 6, 7\}.$$

Thus every target configuration admits a first-occurrence labeling above one of the two roots

$$R_1 = \{\{0, 1, 3\}, \{1, 2, 4\}, \{0, 2, 5\}, \{0, 4, 6\}\}$$

and

$$R_2 = \{\{0, 1, 3\}, \{1, 2, 4\}, \{0, 2, 5\}, \{0, 6, 7\}\}.$$

5 Restricted-growth enumeration and coverage

Fix a point cap M . At every search state, the introduced labels form an initial interval

$$0, 1, \dots, m - 1.$$

The search chooses the least introduced point p whose current degree is less than three. A new block through p is written $\{p, q, r\}$ with $p < q < r$. First-occurrence labeling leaves exactly three possibilities:

$$\begin{aligned} \{p, q, r\} & \quad (p < q < r < m), \\ \{p, q, m\} & \quad (p < q < m), \end{aligned}$$

and

$$\{p, m, m + 1\}.$$

For fixed p , candidates are considered in lexicographic order of (q, r) . A candidate is rejected if a point would exceed degree three, a pair would be repeated, or adding the block would create a forbidden Berge 4- or 8-cycle.

Proposition 5.1 (Restricted-growth coverage). *Let $M \geq 8$. Let $\mathcal{C}$ be a connected linear symmetric v_3 -configuration with $v \leq M$, containing a Berge triangle and no Berge cycle of length 4 or 8. Assume that each cycle-rejection predicate is sound: a rejection occurs only when adding the proposed block really creates the corresponding forbidden Berge cycle. Then one of the two triangle-rooted restricted-growth search trees with cap M contains a completed state isomorphic to $\mathcal{C}$.*

Proof. Normalize a Berge triangle as in the previous section, placing $\mathcal{C}$ above either R_1 or R_2 . We follow a branch consisting only of target blocks from $\mathcal{C}$.

At each stage let $0, \dots, m - 1$ be the introduced target points and let p be the least introduced point whose current degree is less than three. Maintain the following invariants:

- (I1) every target block whose least point is smaller than p has already been inserted;
- (I2) among target blocks whose least point is p , the inserted blocks form a lexicographic initial segment;
- (I3) the introduced target labels are exactly $0, \dots, m - 1$, and unseen target points receive new labels in first-occurrence order.

We make the base case explicit. After the four root blocks have been inserted, point 0 is saturated, so the least deficient point is $p = 1$. The root contains $\{1, 2, 4\}$. The remaining target block through 1 cannot use 0 or 3, because those pairs already occur in $\{0, 1, 3\}$; nor can it use 2 or 4, because those pairs already occur in $\{1, 2, 4\}$. By linearity, if that remaining block is $\{1, q, r\}$ with $q < r$, then $q \geq 5$. Hence $(2, 4) <_{\text{lex}} (q, r)$, proving (I2) at the root. The other two invariants are immediate from the normalized labeling.

Assume the invariants hold at a later state. Since p is deficient, let

$$B = \{p, q, r\}, \quad p < q < r,$$

be the lexicographically first missing target block whose least point is p .

If both q and r are already introduced, the generator proposes the old-old candidate $\{p, q, r\}$. If exactly one is new, first-occurrence labeling gives $\{p, q, m\}$. If both are new, it gives $\{p, m, m + 1\}$. Thus the next target block is always proposed.

Invariant (I2) prevents the lexicographic filter from skipping B . The degree filter cannot reject B , because every point of $\mathcal{C}$ has final degree three. The repeated-pair filter cannot reject B , because $\mathcal{C}$ is linear. Finally, the current partial state together with B is a subconfiguration of $\mathcal{C}$, which contains no forbidden Berge 4- or 8-cycle. Soundness of the two cycle predicates therefore prevents either from rejecting B .

After inserting B , the invariants persist. Induction therefore constructs a branch consisting entirely of target blocks.

It remains to show that every target point is eventually introduced. Suppose all currently introduced target points become saturated while some target point remains unseen. Let S be the nonempty proper set of introduced points. Every target block incident with a point of S has then already been inserted; inserting such a block introduces all its points. Hence every block meeting S is entirely supported on S . The incidences supported on S form a nonempty proper union of components of the Levi graph, contradicting connectedness.

Thus all target points are introduced. The point cap is safe because $v \leq M$ and first-occurrence labels never exceed $v - 1$. The block cap is also safe: a symmetric v_3 -configuration has exactly v blocks. If $v < M$, completion occurs before the cap; if $v = M$, the M th target block yields the complete configuration, and the implementation tests for completion before applying the block-count cutoff. The target branch therefore reaches a completed state isomorphic to $\mathcal{C}$. $\square$

6 Correctness of the cycle-pruning predicates

The fixed search implementation applies the two predicates `creates_C8` and `creates_C16` only after a candidate passes the degree and repeated-pair tests, so adding the candidate preserves linearity. Although Proposition 5.1 requires only soundness, both predicates are exact on such candidates.

6.1 The C_8 predicate

Let t be a structurally admissible candidate block. The predicate enumerates three pairwise distinct old blocks B_i, B_j, B_h and forms the four intersections

$$p_0 = t \cap B_i, \quad p_1 = B_i \cap B_j, \quad p_2 = B_j \cap B_h, \quad p_3 = B_h \cap t.$$

It reports success exactly when all four intersections exist and the four points are pairwise distinct.

Lemma 6.1. *For a structurally admissible candidate t ,*

$$\text{creates_C8}(t) = \text{true}$$

if and only if adding t creates a simple C_8 containing the new block vertex.

Proof. If the predicate succeeds, then after adding the new block vertex T the alternating sequence

$$T, p_0, B_i, p_1, B_j, p_2, B_h, p_3, T$$

has four distinct point vertices and four distinct block vertices, and is therefore a simple C_8 .

Conversely, any new C_8 containing T contains exactly three old block vertices, say B_i, B_j, B_h , together with four distinct intervening point vertices. These are precisely the four intersections above. The implementation enumerates every ordered triple of distinct old blocks, so it finds the witness. $\square$

6.2 The exact length-14 path routine

The C_{16} predicate uses a helper routine that searches the old Levi graph for a simple path of exactly 14 incidence edges between two candidate points. Its visited structure contains both point and block vertices. Reaching the target before depth 14 is forbidden, and success occurs only at depth 14.

Lemma 6.2. *For two distinct old point vertices x, y , the helper succeeds if and only if the old Levi graph contains a simple path of exactly 14 edges from x to y .*

Proof. Every recursive step follows a Levi incidence edge. The visited structure prevents every point or block vertex from appearing twice on an active branch, so a successful branch is a simple path. Success is accepted only at depth 14 at the target.

Conversely, along any simple length-14 path from x to y , the recursive enumeration can choose the path edges in order. No path vertex has been visited previously, and y is not reached early. The routine therefore succeeds at depth 14. $\square$

6.3 The C_{16} predicate

The predicate tests all three unordered pairs of points in the candidate block t using the exact-path routine.

Lemma 6.3. *For a structurally admissible candidate t ,*

$$\text{creates_C16}(t) = \text{true}$$

if and only if adding t creates a simple C_{16} containing the new block vertex.

Proof. If two members $x, y \in t$ are joined by a simple old path of length 14, then adding the new block vertex T and the edges Tx, Ty closes this path into a simple C_{16} .

Conversely, delete T and its two cycle edges from any new simple C_{16} containing T . What remains is a simple old path of length 14 joining the two members of t used by the cycle. Since the predicate tests all three pairs, it finds this path. $\square$

The third member of t may occur internally on the old path. The third incidence edge from T is then merely a chord; a simple cycle need not be induced. Consequently the predicate is still correct.

Lemmas 6.1 and 6.3 in particular imply the no-false-positive property required in Proposition 5.1.

7 The cap-34 finite exhaustion

We now set $M = 34$. The two rooted searches terminate with the statistics in Table 1.

Root	States	Attempted	Structural	C_8	C_{16}	Completions
R_1	23,143	2,972,750	180,011	1,424,785	1,344,812	0
R_2	202,602	25,227,209	1,432,521	15,207,779	8,384,308	0
Total	225,745	28,199,959	1,612,532	16,632,564	9,729,120	0

Table 1: Cap-34 triangle-rooted search statistics.

For each root,

$$\text{attempted} = \text{structural} + C_8 + C_{16} + \text{expanded}, \quad \text{expanded} = \text{states} - 1.$$

These identities are useful consistency checks, but they do not by themselves establish exhaustive execution. For that purpose we use a static certificate.

8 Static certification of the finite search

The certificate verifier reconstructs the restricted-growth proposal schedule from each current state rather than accepting a candidate list supplied by the certificate generator. For every expected proposal exactly one of four outcomes must be verified:

1. a directly checked structural rejection;
2. rejection with an explicit C_8 witness;
3. rejection with an explicit C_{16} witness;
4. acceptance and recursive verification of the resulting child state.

For a C_8 rejection, the certificate supplies three old block identifiers. The verifier reconstructs the four intersections and checks that they form a simple C_8 through the candidate. For a C_{16} rejection, the certificate supplies a pair of candidate points and seven old block identifiers; the verifier checks an alternating simple old path of length 14 and hence a C_{16} after the candidate is added. For an accepted candidate, the verifier constructs the child state itself and continues recursively.

The critical point is that the verifier independently knows which proposal is next. A certificate cannot silently omit a legal proposal: missing, reordered, malformed, or prematurely terminated certificate data causes verification to fail.

Proposition 8.1 (Certificate exhaustion). *Fix one of the two cap-34 rooted search trees. Suppose the certificate verifier reconstructs the complete ordered candidate list at every state, checks every rejection witness, recursively verifies every accepted child, detects completed states, and rejects missing or inconsistent certificate data. If the verifier accepts the complete stream and reports zero solutions, then the rooted search tree contains no completed state.*

Proof. At a verified state S , let $t_1, \dots, t_k$ be the complete ordered candidate list reconstructed by the verifier. For each t_i , either the verifier establishes a valid structural or forbidden-cycle reason closing that branch, or it constructs the exact child $S + t_i$ and recursively verifies its subtree. Thus every child branch of S is either validly closed or recursively exhausted. The cap makes the tree finite, so recursion terminates. Every completed state encountered by this traversal is counted. Therefore an accepted complete stream with zero solutions certifies that no completion occurs anywhere in the rooted tree. $\square$

The two fixed certificate streams are both accepted. The verifier reports

$$\text{solutions} = 0$$

for both roots. Their compressed SHA-256 digests are

$$\begin{aligned} R_1 : & \text{dbc3b405bff9cadba5687e2eb8b6ba033255fc99d23b4bf7f49efc4d6e8b79a,} \\ R_2 : & \text{f8cd411dc6b26b5a703df811ad53c055d1e7ea75dbca2704a52bf0bbea911bd0.} \end{aligned}$$

9 Proof of the main theorem

Proof of Theorem 1.1. Suppose that a simple cubic bipartite graph G with $|V(G)| \leq 68$ contains no C_4 , C_8 , or C_{16} . Choose any connected component H of G . Since G is cubic and bipartite, so is H , and $|V(H)| \leq 68 < 70$.

By Lemma 3.1, H contains a C_6 . Construct its incidence configuration $\mathcal{C}$. Since H is C_4 -free, $\mathcal{C}$ is linear. The C_6 is a Berge triangle, while the absence of C_8 and C_{16} says that $\mathcal{C}$ has no Berge 4- or 8-cycle. Because H is connected cubic bipartite,

$$|V(H)| = 2v$$

for a connected linear symmetric v_3 -configuration with $v \leq 34$.

By Proposition 5.1, a restricted-growth labeling of $\mathcal{C}$ must occur as a completed state in one of the two cap-34 triangle-rooted search trees. Lemmas 6.1 and 6.3 ensure that the target branch cannot be falsely pruned by the cycle tests. But Proposition 8.1, together with the two accepted static certificate streams, shows that neither rooted tree contains a completion. This contradiction proves the theorem. $\square$

Corollary 1.2 follows because every cubic bipartite graph has even order.

10 Reproducibility and provenance

The reference search implementation is the triangle-rooted program distributed with Tranquilli [3]. The direct cap-34 reproduction is anchored to the immutable upstream commit

67adc92d31d5d1edfbd0a7b845dc232e31c412cd.

A fresh Git clone was checked out at this exact commit. The working tree was clean, the upstream manifest verification succeeded, and the SHA-256 digest of the relevant upstream file `research/src/triangle_root_universal_search.cpp` was

e778648bc3d5b0a19c860bec1371a542c464652693c2cbc4c2cd5ef05f314f41.

The source was compiled directly from this fresh checkout with `-DSIDE=34`. The resulting run reproduced all search counts in Table 1 and the decision-transcript hashes

R_1 : db548be2919a7f8a8b28c5ded67aa622779dd3e22192ee8a2a1e065f8c07b4c5,

R_2 : c2e03a0f97ea3c649a5feaf4d4145b229da6f56aa31b8ebdb8b769b18848607.

The static-certificate generator and verifier used here are project-specific additions, not upstream components. The complete artifact was independently verified in a fresh Ubuntu 24.04 environment with GCC/g++ 13.3.0. The verification script checked the artifact manifest, rebuilt the verifier, accepted both certificate streams with the exact expected counters, reported zero solutions for both roots, and terminated with `ARTIFACT VERIFIED`.

The packaged artifact has SHA-256 digest

32558c65679736fb2ff75b67b2d75a85a2b7b91e6dde5d15b06c3a9b59f0d7de.

Source code, certificates, build instructions, expected output, tamper tests, and provenance records are intended to accompany the public version of this manuscript.

11 Trust boundary

The static certificate substantially reduces the amount of computation that must be trusted. In particular, the conclusion does not rest only on a depth-first-search program printing a zero-solution summary: the verifier reconstructs the proposal schedule and checks explicit local reasons for rejected branches.

The computation is nevertheless not formally verified in a proof assistant. The remaining conventional trusted computing base includes the verifier source, the correctness of its compilation, the C++ implementation and compiler, the operating system and hardware, and SHA-256 for artifact identity. The justified claim is therefore that the finite exhaustion is independently executable and certificate-checked, not that the entire software stack has been formally verified from first principles.

AI-assisted development disclosure

Generative AI tools were used during exploratory reasoning, literature review, source-code interpretation, proof auditing, adversarial review, artifact design, and manuscript drafting. AI-generated output is not used as an evidentiary substitute for the mathematical proof or computational certificate. The theorem-supporting claims are reduced to explicit mathematical arguments, immutable source identity, reproducible computations, static certificate data, and independently executable verification. Responsibility for the mathematical statements and any released manuscript or artifact rests with the human author.

12 Concluding remarks

Theorem 1.1 establishes a 70-vertex lower bound for cubic bipartite counterexamples to the Erdős–Gyárfás conjecture. The number 70 enters structurally: below 70, exclusion of C_4 and C_8 forces a C_6 , allowing the entire problem to be rooted at a Berge triangle. At order 70, cubic graphs of girth 10 exist, so this forcing step no longer applies. The present argument therefore deliberately stops before the 70-vertex case.

Further progress would require either additional structure for the girth-10 case or an enumeration framework that does not rely on a forced triangle. No such extension is claimed here.

References

- [1] P. Erdős, Some old and new problems in various branches of combinatorics, *Discrete Mathematics* **165–166** (1997), 227–231.
- [2] M. O’Keefe and P. K. Wong, A smallest graph of girth 10 and valency 3, *Journal of Combinatorial Theory, Series B* **29** (1980), 91–105.
- [3] J. Tranquilli, A 60-vertex lower bound for cubic bipartite counterexamples to the Erdős–Gyárfás conjecture, arXiv:2608.02675, 2026.